Video Streaming

renditions, segments, a player that adapts

Watch the 4-min video

> **In one line:** transcode each upload into a ladder of renditions, cut each into short keyframe-aligned segments listed in an HLS or DASH manifest, and let the player pick the rendition for each next segment from measured bandwidth and buffer; segments are ordinary immutable files, so a CDN serves almost all of the traffic.

1 One file can't fit everyone

A creator uploads a 2 GB 4K video. A viewer on a train with a weak signal presses play. Send the original (say 40 Mbit/s) and it buffers forever; send one small 480p file to everyone and viewers on fibre and big TVs get a blurry picture. Networks also change minute to minute, and devices differ in codecs and screen sizes. Video needs **several versions** and the ability to **switch mid-stream** without stopping.

2 Transcoding into renditions

The original is decoded and re-encoded into a **bitrate ladder**: the same video at several resolutions and bitrates. An illustrative H.264 ladder:

Rendition	Resolution	Video bitrate (example)	10 minutes ≈
240p	426 × 240	0.4 Mbit/s	30 MB
360p	640 × 360	0.8 Mbit/s	60 MB
480p	854 × 480	1.4 Mbit/s	105 MB
720p	1280 × 720	2.8 Mbit/s	210 MB
1080p	1920 × 1080	5 Mbit/s	375 MB

All five together are about 10.6 Mbit/s, so a 10-minute video costs ~800 MB of renditions on top of the original. **Codecs**: H.264/AVC plays everywhere; HEVC, VP9 and AV1 typically need 30–50% less bitrate for the same quality but cost much more CPU to encode and aren't supported on every device, so platforms encode popular videos in newer codecs too and let the player choose. **Per-title encoding** (Netflix, 2015) tunes the ladder to each video's complexity: a cartoon needs far fewer bits than a football match.

3 Segments and manifests: HLS and DASH

```
# master.m3u8  (one line per rendition)
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=5300000,RESOLUTION=1920x1080,CODECS="avc1.640028,mp4a.40.2"
1080p/index.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1280x720,CODECS="avc1.64001f,mp4a.40.2"
720p/index.m3u8
# 720p/index.m3u8  (media playlist: the segments, in order)
#EXTM3U
#EXT-X-TARGETDURATION:6
#EXTINF:6.0,
seg_00001.m4s
#EXTINF:6.0,
seg_00002.m4s
#EXT-X-ENDLIST
```

Figure 1. HLS: a master playlist lists renditions (bandwidth, resolution, codecs); each media playlist lists that rendition's segments in order.

Each rendition is cut into **segments** of typically 2–6 seconds (Apple's HLS guidance uses 6 s). For seamless switching, every segment must start with a keyframe (IDR) and keyframes must be **aligned across all renditions** (fixed GOP, closed GOPs), so segment 37 covers exactly the same time in every rendition. A **manifest** describes it all:

	HLS	MPEG-DASH
Origin	Apple (RFC 8216)	ISO/IEC 23009-1 standard
Manifest	`.m3u8` playlists (master + media)	`.mpd` XML
Segments	MPEG-TS or fragmented MP4	fragmented MP4 (or WebM)
Playback	native on Apple devices; hls.js elsewhere	dash.js / Shaka Player via Media Source Extensions; Android ExoPlayer

CMAF standardises fragmented-MP4 segments that both HLS and DASH can reference, so you store the media once and publish two small manifests.

4 How adaptive bitrate switching works

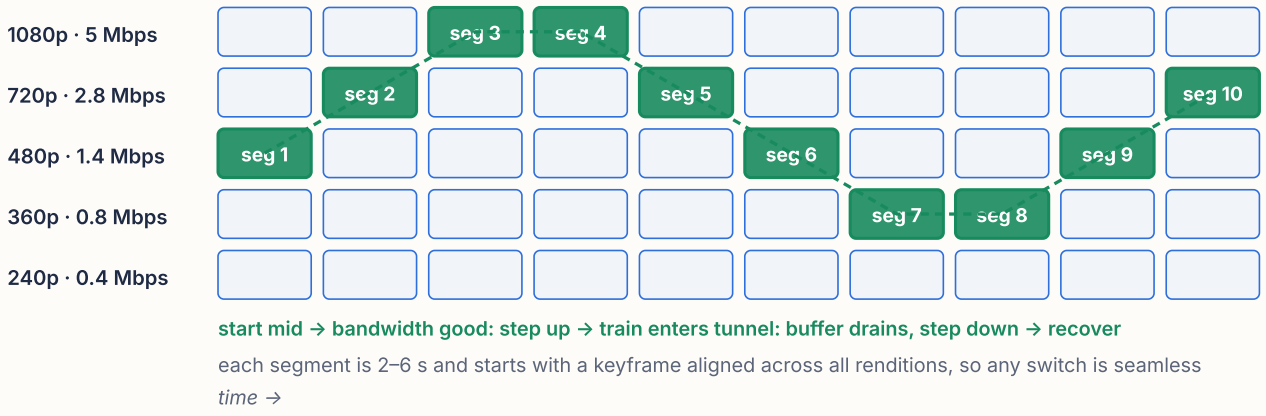


Figure 2. The player chooses a rendition per segment. Here it starts at 480p, climbs to 1080p, drops to 360p as the buffer drains, then recovers.

1. **Start**: fetch the master manifest, pick a modest rendition so the first frame appears quickly (start-up time matters more than initial sharpness), and fill a buffer.

2. **Measure**: after each segment, estimate throughput (segment bytes ÷ download time, smoothed) and check the buffer level.
3. **Decide** for the next segment: *throughput-based* rules pick the highest bitrate below a safety fraction (e.g. 70–80%) of the estimate; *buffer-based* rules (BBA, Netflix/Stanford, SIGCOMM 2014; BOLA in dash.js) map buffer level to bitrate, stepping down as the buffer drains; hybrids combine both. Switching up is cautious, switching down is fast, to avoid stalls.
4. **Switch at a boundary**: the next segment simply comes from another media playlist. Playback never stops, and a rebuffer only happens if the buffer empties completely.

The quality metrics that platforms optimise: start-up time, rebuffer ratio, average bitrate, and the number of switches.

5 Processing as a DAG

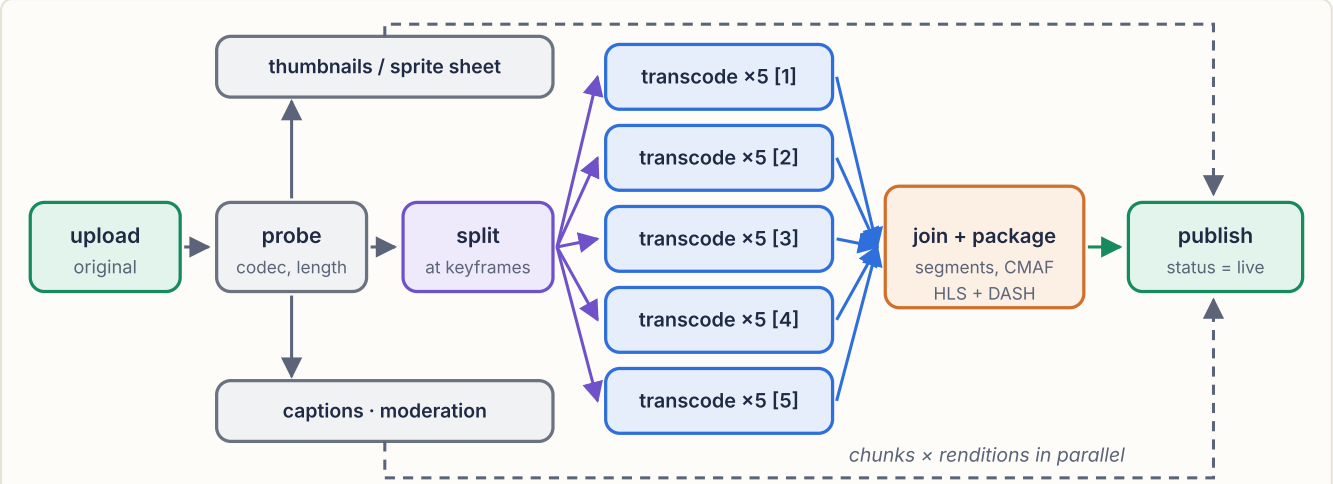


Figure 3. A YouTube-style processing DAG: split, transcode chunks in parallel, join and package, with side branches. Each node retries independently.

Transcoding a 2-hour 4K film on one machine can take hours. Pipelines split the work: **probe** the input (codec, duration, rotation), **split** it into chunks at keyframes (e.g. 30–60 s), **transcode** every chunk × rendition as an independent task on a worker pool, then **join** and **package** into segments and manifests. A 10-minute video in 20 chunks × 5 renditions is 100 parallel tasks and finishes in minutes. Thumbnails, captions (speech-to-text) and content moderation run as separate branches. An orchestrator (Step Functions, Temporal, Airflow, or a home-grown DAG engine) tracks dependencies, retries failed tasks only, and marks the video **live** when the required nodes finish; high-resolution or new-codec renditions can be added later, after the video is already watchable.

6 Delivery via CDN

Segments are ordinary files that never change once written, so they get long cache lifetimes; VOD manifests are cacheable too (live manifests only for about one segment duration). A popular video is served almost entirely from edge caches; the origin sees roughly one request per segment per region (fewer with an origin shield, lesson 11). The hard part is the **long tail**: millions of rarely watched videos miss the cache and hit storage. Large platforms push caches into ISPs: Netflix's Open Connect appliances sit inside ISP networks and are filled overnight with what is predicted to be popular. Paid content adds signed cookies (lesson 11) and DRM (Widevine, FairPlay, PlayReady, via Common Encryption). Live streaming adds ingest (RTMP or SRT), real-time transcoding and constantly updated manifests; Low-Latency HLS and chunked CMAF bring latency down to a few seconds.

7 Example: YouTube-style upload → transcode → CDN

WORKED EXAMPLE: A 10-MINUTE 4K UPLOAD

The creator uploads a 2 GB original to object storage with a resumable multipart upload (lesson 6). The `ObjectCreated` event starts the DAG: probe (3840 × 2160, H.264, 10:00), split into 20 thirty-second chunks, transcode 20 × 5 = 100 tasks in parallel into 240p–1080p H.264, join, cut into 6-second segments (100 per rendition, 500 in total) as CMAF, write HLS and DASH manifests. A side branch extracts thumbnails. When the required nodes finish, the video row becomes `live`, about 800 MB of renditions sit next to the original, and viewers stream from the CDN. The train passenger starts at 360p and moves between 240p and 720p as the signal changes; the fibre viewer settles on 1080p within a few segments.

TRADE-OFFS

- More renditions: smoother adaptation, but more encoding CPU and storage.
- Short segments: faster switching and lower live latency, but more requests and slightly worse compression (more keyframes).
- Newer codecs: big bandwidth savings, but costly encoding and partial device support; often worth it only for popular titles.
- Encoding everything up front vs on first view: instant availability vs saving CPU on videos nobody watches.

COMMON MISTAKES

- Keyframes not aligned across renditions, so switches glitch or stall.
- Transcoding a long video as one serial job: hours of latency and a full restart on failure.
- Short CDN TTLs on immutable segments, sending traffic back to the origin.
- Starting playback at the highest rendition, making start-up slow.

8 Key terms

Rendition / bitrate ladder: the same video at several resolutions and bitrates

Manifest: HLS .m3u8 or DASH .mpd listing renditions and segments

CMAF: common fMP4 segment format usable by both HLS and DASH

Segment: short media file (2–6 s) starting with a keyframe

ABR: adaptive bitrate: the player picks a rendition per segment

Per-title encoding: a ladder tuned to each video's complexity

9 Interview questions

Q1. How does adaptive bitrate streaming work? BASIC

The video is transcoded into renditions, each cut into short segments listed in a manifest; the player measures bandwidth and buffer and picks the best rendition for each next segment, switching without stopping.

Q2. Why cut video into segments? BASIC

Switches happen at segment boundaries, playback can start after one segment, and segments are ordinary immutable files any CDN can cache.

Q3. Transcoding a 2-hour upload takes 3 hours. How do you speed it up? SCENARIO
Split at keyframes into chunks and transcode chunks × renditions in parallel across workers, then join; publish lower renditions first and add the rest later.

Q4. Viewers complain of frequent quality flips. What would you look at? SCENARIO
The ABR rules: smoothing of the throughput estimate, safety margin, buffer thresholds and hysteresis for switching up; also whether segments are too short.

Q5. How would you estimate storage for 1 million hours of video? SCENARIO
Ladder sum × duration: ~10.6 Mbit/s ≈ 4.8 GB per hour of renditions, so ~4.8 PB, plus originals and any extra codecs.

Q6. Why must keyframes be aligned across renditions? TRICKY
So that segment N covers the same time span and starts decodable in every rendition; otherwise switching mid-stream shows glitches or needs extra decoding.

Q7. The CDN hit ratio is 98% but origin load is still high. Why? TRICKY
The long tail: many rarely watched videos and their many segments miss at each edge; add an origin shield, tiered caching, and pre-positioning of predicted-popular content.

Next: SD-O-011 · CDN for Private Content backendroadmap.com